

SOFTWARE DESIGN DOCUMENT (SDD)

PRESENTED BY: SoftAlchemy Sdn. Bhd.	PRESENTED TO: Malaysian Communication and Multimedia Commission (MCMC)	DATE: 09 May 2025
---	--	---------------------------------

TEAM MEMBERS

Chief Executive Officer	Chief Business Analyst	Chief Developer	Software Quality Assurance
			
SITI NAJWA BINTI BAKRI (CB23013)	NOORA HAIDAH BINTI AZMI (CB23095)	NUR WAHIDAH SYARINI BINTI ZALAHUDIN (CB23024)	NOORA HANIM BINTI AZMI (CB23040)

Submitted to the Software Design Workshop (BCS2343)
Bachelor of Computer Science (Software Engineering) with Honours
Faculty of Computing, Universiti Malaysia Pahang Al-Sultan Abdullah (UMPSA)

DOCUMENT APPROVAL

	Name	Date
Authenticated by: _____ Project Leader		
Approved by: _____ Project Manager		
Approved by: _____ Client		

Software: PDF

Archiving Place:

https://drive.google.com/drive/folders/1Xx0ulE7F8QdFEaZqhJqt4bStanKOn7Oc?usp=drive_link

Copies Available: 1

TABLE OF CONTENT

TABLE OF CONTENT	3
LIST OF FIGURES	5
LIST OF ABBREVIATIONS	6
1. INTRODUCTION	3
1.1 Purpose.....	3
1.2 System Identification	4
1.3 System Overview	7
1.4 References.....	9
2. DATA DESIGN.....	10
2.1 Entity Relationship Diagram (ERD)	10
2.2 DATA DICTIONARY	11
2.2.1 Table User.....	11
2.2.2 Table PublicUser.....	11
2.2.3 Table MCMCStaff	12
2.2.4 Table Agency	12
2.2.2 Table Inquiry.....	13
3. GENERAL ARCHITECTURE.....	14
3.1 APPLICATION LAYER	14
3.2 BUSINESS SERVICES LAYER.....	17
3.2.1 Controller [SDD-REQ-200].....	18
3.2.2 Models [SDD-REQ-300]	18
3.3 MIDDLEWARE LAYER	19
3.4 PACKAGE RELATIONSHIP	20
4. DETAIL DESIGN	21
4.1 MODEL [SDD_MYS_100].....	21
4.1.1 User Model [SDD_MYS_100_101]	21
4.1.2 PublicUser Model [SDD_MYS_100_102]	22
4.1.3 McmcStaff Model [SDD_MYS_100_103]	22
4.1.4 Agency Model [SDD_MYS_100_104].....	23
4.1.5 ActivityLog Model [SDD_MYS_100_105].....	23
4.1.6 InquiryModel [SDD_MYS_100_201]	23
4.2 VIEW [SDD_MYS_200].....	27
4.2.1 MANAGE USER [SDD_MYS_200_100].....	27
4.2.2 MANAGE INQUIRY FORM SUBMISSION [SDD_MYS_200_200]	34
4.2.3 MANAGE INQUIRY ASSIGNMENT [SDD_MYS_200_300].....	43
4.2.4 MANAGE PROGRESS TRACKING [SDD_MYS_200_400]	51
4.3 CONTROLLER [SDD_MYS_300].....	56
4.3.1 MANAGE USER [SDD_MYS_300_100].....	56
4.3.2 MANAGE INQUIRY FORM SUBMISSION [SDD_MYS_300_200]	61

4.3.3	MANAGE INQUIRY ASSIGNMENT [SDD_MYS_300_300].....	65
4.3.4	MANAGE PROGRESS TRACKING [SDD_MYS_300_400]	68
5.	REQUIREMENT TRACEABILITY.....	71
5.1	MANAGE USER.....	71
5.2	MANAGE INQUIRY FORM SUBMISSION	73
5.3	MANAGE INQUIRY ASSIGNMENT	74
5.4	MANAGE PROGRESS TRACKING	75

LIST OF FIGURES

Figure 2.1 Entity Relationship Diagram MySebenarnya System	10
Figure 3.1 Application Layer for MySebenarnya System	14
Figure 3.2 Business Services Layer for MySebenarnya System	17
Figure 3.3 Middleware Layer for MySebenarnya System	19
Figure 3.4 Model View Controller (MVC) Package Relationship for MySebenarnya Syatem	20

LIST OF ABBREVIATIONS

1. SDD - Software Design Document
2. MYS – MySebenarnya
3. UC – Use Case
4. REQ – Requirement
5. KKM – Kementerian Kesihatan Malaysia
6. JPJ – Jabatan Pengangkutan Jalan

1. INTRODUCTION

1.1 Purpose

The purpose of this document is to clearly describe the design of MySebenarnya system, which is developed to combat the spread of misinformation by allowing users to submit new inquiries for verification. This document outlines the overall architecture of the MySebenarnya system, including its key components, relationships and the interfaces between them. Moreover, this document provides detailed descriptions of the system's functionalities and design decisions. By documenting the architectural and detailed design of the system, the SDD will serve as a guide to developers, designers and stakeholders as to what design will be developed for the MySebenarnya system, and how it will be implemented.

1.2 System Identification

System Title	MySebenarnya System						
System Abbreviations	MYS						
System Identification Number	MYS_2025_V01 This system identification number shows the identification number for the MySebenarnya System. Meanings for terms use: <table><tr><td>MYS</td><td>MySebenarnya (System Name)</td></tr><tr><td>2025</td><td>Year document created</td></tr><tr><td>V01</td><td>Version number</td></tr></table>	MYS	MySebenarnya (System Name)	2025	Year document created	V01	Version number
MYS	MySebenarnya (System Name)						
2025	Year document created						
V01	Version number						
Version Number	MYS_2025_V01 This is the system initial version for the MySebenarnya System to the user. Meanings for terms use: <table><tr><td>MYS</td><td>MySebenarnya (System Name)</td></tr><tr><td>2025</td><td>Year the system released</td></tr><tr><td>V01</td><td>Version number</td></tr></table>	MYS	MySebenarnya (System Name)	2025	Year the system released	V01	Version number
MYS	MySebenarnya (System Name)						
2025	Year the system released						
V01	Version number						

Document Identification Number	SDD_MYS_2025_V01 This system identification number shows the identification number for the MySebenarnya System. Meanings for terms use: <table> <tr> <td>SDD</td><td>Software Design Document</td></tr> <tr> <td>MYS</td><td>MySebenarnya (System Name)</td></tr> <tr> <td>2025</td><td>Year document created</td></tr> <tr> <td>01</td><td>Version number</td></tr> </table>	SDD	Software Design Document	MYS	MySebenarnya (System Name)	2025	Year document created	01	Version number
SDD	Software Design Document								
MYS	MySebenarnya (System Name)								
2025	Year document created								
01	Version number								
Package Number	SDD_MYS_100 Meanings for terms use: <table> <tr> <td>SDD</td><td>Software Design Document</td></tr> <tr> <td>MYS</td><td>MySebenarnya (System Name)</td></tr> <tr> <td>100</td><td>Package Number</td></tr> </table>	SDD	Software Design Document	MYS	MySebenarnya (System Name)	100	Package Number		
SDD	Software Design Document								
MYS	MySebenarnya (System Name)								
100	Package Number								
Module ID	SDD_MYS_100_100 Meanings for terms use: <table> <tr> <td>SDD</td><td>Software Design Document</td></tr> <tr> <td>MYS</td><td>MySebenarnya (System Name)</td></tr> <tr> <td>100</td><td>Package Number</td></tr> <tr> <td>100</td><td>Module Number</td></tr> </table>	SDD	Software Design Document	MYS	MySebenarnya (System Name)	100	Package Number	100	Module Number
SDD	Software Design Document								
MYS	MySebenarnya (System Name)								
100	Package Number								
100	Module Number								
Class ID	SDD_MYS_100_101 Meanings for terms use: <table> <tr> <td>SDD</td><td>Software Design Document</td></tr> <tr> <td>MYS</td><td>MySebenarnya (System Name)</td></tr> <tr> <td>100</td><td>Package Number</td></tr> <tr> <td>100</td><td>Module Number</td></tr> </table>	SDD	Software Design Document	MYS	MySebenarnya (System Name)	100	Package Number	100	Module Number
SDD	Software Design Document								
MYS	MySebenarnya (System Name)								
100	Package Number								
100	Module Number								

	01	Class Number
Requirement ID	REQ_UC01_001	
	Meanings for terms use:	
	REQ	Requirement
	UC01	Use Case Number in this document
	001	Requirement Number in this document
Design ID	SDD_MYS_2025_101	
	Meanings for terms use:	
	SDD	Software Design Document
	MYS	MySebenarnya (System Name)
	2025	Year document created
	100	Module Number
	01	Design Number in this document

1.3 System Overview

The system to be developed shall provide facilities to manage and verify public complaints related misinformation, serving the operational needs of the Malaysian Communications and Multimedia Commission (MCMC). The system consists of the following modules:

1. Module 1: Manage User

The Manage User module handles user authentication and role-based access control. Users are divided into three categories which are public users, MCMC Staff, and agency representatives with specific access rights and responsibilities. Core features such as account creation, login, profile management, password recovery, and activity logging are included in this module to ensure user accountability. MCMC Staff have access to additional functionalities, such as user access data and generating reports based on data.

2. Module 2: Manage Inquiry Form Submission

The Manage Inquiry Form Submission module is designed to provide public users with a user-friendly interface for reporting unverified news or suspected misinformation by entering relevant details, uploading supporting documents, and specifying their inquiry. Public users can submit the inquiry, view all past submitted inquiries and view all inquiries submitted by other individuals. While MCMC Staff can access new and previous inquiries submitted from public users, inquiry filtering and validation and generate reports. The agency can view a record of all inquiries that are assigned to them and view the full history of each inquiry.

3. Module 3: Inquiry Assignment

The inquiry assignment module is for MCMC Staff to assign the inquiries to the relevant agencies automatically or manually based on the subject matter. Using predefined classification criteria, the system can direct specific inquiries to relevant authorities. Public users can only see the inquiry assigned to each agency. The Agency will receive inquiries assigned by MCMC for verification, jurisdiction review and can reject an inquiry by providing reason and notifying MCMC.

4. Module 4: Inquiry Progress Tracking

The inquiry progress tracking module enables efficient tracking and management of complaint statuses across the system. Each inquiry is assigned a unique tracking ID and is updated in real-time as it moves through various stages. Public users have visibility into the status of their inquiries and receive notification when the status changes, while MCMC Staff can view and track the progress of inquiries assigned to all agencies and generate reports. The agency must update each status of inquiry that assigned by MCMC Staff by providing the supporting documents and explanation.

1.4 References

- (i) Azma, A., Al-Fahim, M. A., Ku Saimah, K. I., (2024). BCS2233-2024/2025-I Appendix 1 SRS Request for Proposal. Faculty of Computing, Universiti Malaysia Pahang Al-Sultan Abdullah
- (ii) Azma, A., Al-Fahim, Rohani, (2025). BCS2243-2024/2025-II Request For Proposal. Faculty of Computing, University Malaysia Pahang Al-Sultan Abdullah.
- (iii) Guide to the Software Engineering Body of Knowledge (SWEBOK). (2014). *Software design*. In P. Bourque & R. E. Fairley (Eds.), *Guide to the software engineering body of knowledge (SWEBOK), Version 3.0* (pp. 73–106). IEEE Computer Society.
<https://www.computer.org/education/bodies-of-knowledge/software-engineering>
- (iv) Atlassian. (n.d.). How to write a software design document (SDD). Atlassian.
<https://www.atlassian.com/agile/software-development/software-design-document>
- (v) Microsoft. (n.d.). Software design document: What is it & how to create one. Microsoft Learn. <https://learn.microsoft.com/en-us/devops/deliver/design-software>

2. DATA DESIGN

2.1 Entity Relationship Diagram (ERD)

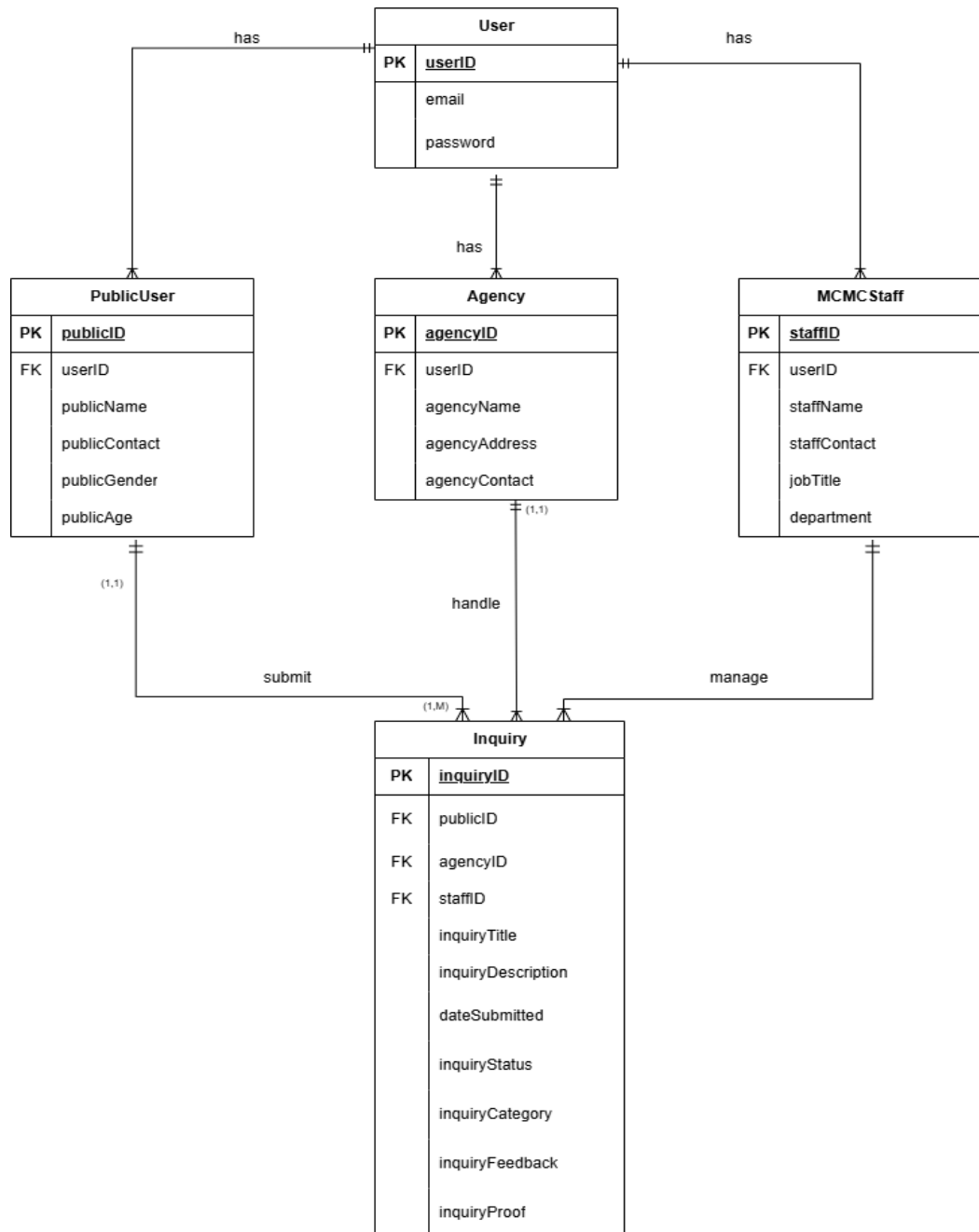


Figure 2.1 Entity Relationship Diagram MySebenarnya System

2.2 DATA DICTIONARY

2.2.1 Table User

Field Name	Description	Data Type	Constraint
userID	Unique identifier for the user	VARCHAR(5)	PK
email	Username of the user	VARCHAR(20)	
password	Password of the user	VARCHAR(12)	

2.2.2 Table PublicUser

Field Name	Description	Data Type	Constraint
publicID	Unique identifier for public users	VARCHAR(5)	PK
userID	Foreign key referencing the Table User	VARCHAR(5)	FK
publicName	Name of the public user	VARCHAR(100)	
publicContact	Phone number of the public user	VARCHAR(11)	
publicGender	Gender of the public user	VARCHAR(6)	
publicAge	Age of the public user	INT	

2.2.3 Table MCMCStaff

Field Name	Description	Data Type	Constraint
staffID	Unique identifier for MCMC Staff	VARCHAR(5)	PK
userID	Foreign key referencing the Table User	VARCHAR(5)	FK
staffName	Name of the MCMC Staff	VARCHAR(100)	
staffContact	Phone number of the MCMC Staff	VARCHAR(11)	
jobTitle	Job title of the MCMC Staff	VARCHAR(20)	
department	Department of the MCMC Staff	VARCHAR(20)	

2.2.4 Table Agency

Field Name	Description	Data Type	Constraint
agencyID	Unique identifier for the agency	VARCHAR(5)	PK
userID	Foreign key referencing the Table User	VARCHAR(5)	FK
agencyName	Name of the agency	VARCHAR(50)	
agencyAddress	Email of the agency	VARCHAR(50)	
agencyContact	Phone number of the agency	VARCHAR(11)	

2.2.2 Table Inquiry

Field Name	Description	Data Type	Constraint
inquiryID	Unique identifier for the inquiry submitted	VARCHAR(5)	PK
publicID	Foreign key referencing the Table PublicUser	VARCHAR(5)	FK
agencyID	Foreign key referencing the Table Agency	VARCHAR(5)	FK
staffID	Foreign key referencing the Table MCMCStaff	VARCHAR(5)	FK
inquiryTitle	Title of the news inquiry	VARCHAR(100)	
inquiryDescription	Description of the inquiry	VARCHAR(200)	
dateSubmitted	Date submitted if the inquiry	DATE	
inquiryStatus	Status of the inquiry	VARCHAR(20)	
inquiryCategory	Category of the inquiry	VARCHAR(20)	
inquiryFeedback	Feedback of the inquiry	VARCHAR(200)	
inquiryProof	Image or documents of the news inquiry	BLOB	

3. GENERAL ARCHITECTURE

3.1 APPLICATION LAYER

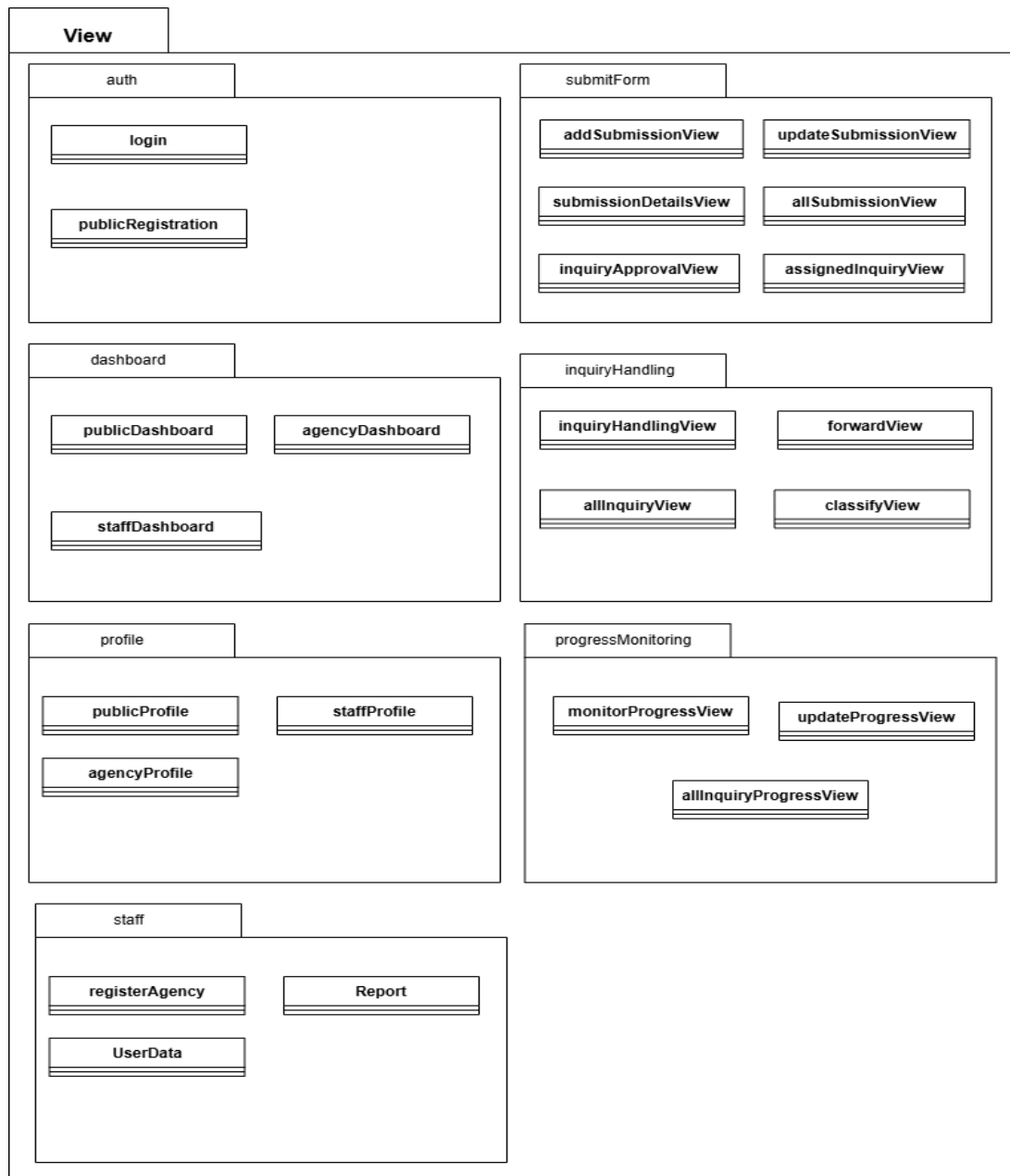


Figure 3.1 Application Layer for MySebenarnya System

3.1.1 View [SDD-REQ-100]

3.1.1.1 Auth [SDD-REQ-101]

Auth	Description
login	This view for displaying the login form and handles user login
PublicRegistration	This view is for public user registration before login into the

3.1.1.2 Dashboard [SDD-REQ-102]

Dashboard	Description
publicDashboard	This homepage view for Public Users after login
agencyDashboard	This homepage view for Agency Representatives after login.
staffDashboard	This homepage view for MCMC Staff after login.

3.1.1.3 Profile [SDD-REQ-103]

Profile	Description
publicProfile	This view for displaying public user profile information.
agencyProfile	This view for displaying agency profile information.
staffProfile	This view for displaying mcmc staff profile information.

3.1.1.3 MCMCStaff [SDD-REQ-104]

Admin	Description
RegisterAgency	This view for MCMC Staff register new agency into the system.
report	This view for displaying reports for MCMC Staff analysis.
userData	This view for displaying all users in the system information.

3.1.1.4 SubmitForm [SDD-REQ-104]

SubmitForm	Description
addSubmissionView	This view for users to submit a new fake news report.
updateSubmissionView	This view for users to edit or update an existing submission.
deleteSubmissionView	This view for users or authorized roles to delete a submission.
submissionDetailsView	This view for displaying detailed information about a specific submission.
allSubmissionsView	This view for displaying a complete list of all submissions made in the system.

3.1.1.5 InquiryHandling [SDD-REQ-105]

InquiryHandling	Description
inquiryHandlingView	This view for displaying incoming inquiries submitted by users.
forwardView	This view for staff to forward inquiries to relevant departments or agencies.
classifyView	This view for classification of inquiries based on predefined categories.
allInquiryView	This view for displaying a list of all inquiries received in the system.

3.1.1.6 ProgressMonitoring [SDD-REQ-106]

ProgressMonitoring	Description
monitorProgressView	This view for displaying status and updates of fake news case investigations.
updateProgressView	This view for authorized users to update complaint progress.
allInquiryProgressView	This view for displaying status of the submitted inquiry.

3.2 BUSINESS SERVICES LAYER

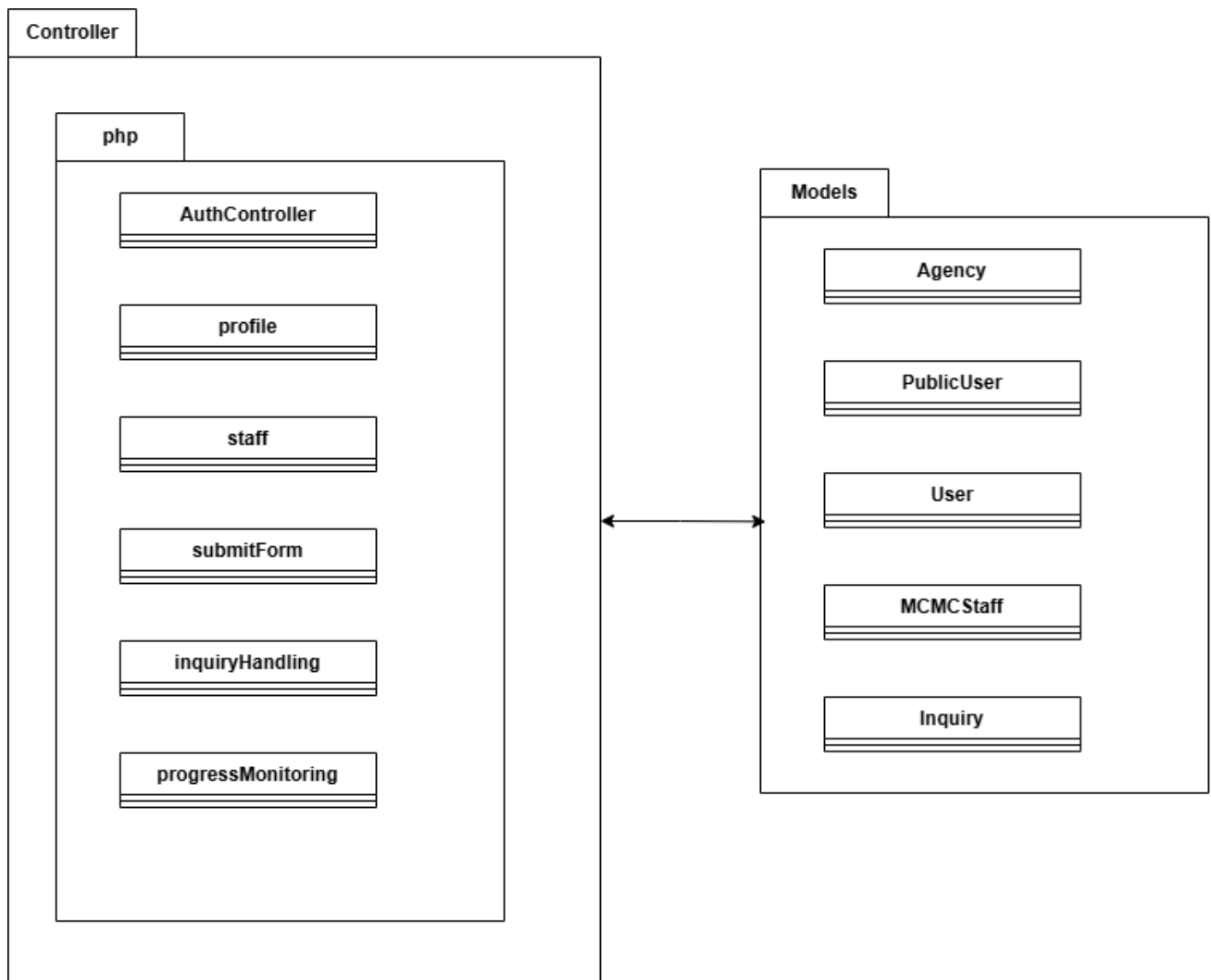


Figure 3.2 Business Services Layer for MySebenarnya System

3.2.1 Controller [SDD-REQ-200]

3.2.1.1 php

Class Name	Description
AuthController	This controller is used to manage user login and registration
profile	This controller is used to manage registration interface
staff	This controller is used to manage staff additional access tools
submitForm	This controller is used to manage public user form submission interface
progressMonitoring	This controller is used to manage inquiry progress monitoring interface

3.2.2 Models [SDD-REQ-300]

Class Name	Description
Agency	To store and retrieve detailed information of Agency in database
PublicUser	To store and retrieve detailed information of Agency in database
User	To store and retrieve detailed information of user username and password in database
MCMCStaff	To store and retrieve detailed information of MCMCStaff in database
Inquiry	To store and retrieve detailed information of inquiry in database

3.3 MIDDLEWARE LAYER

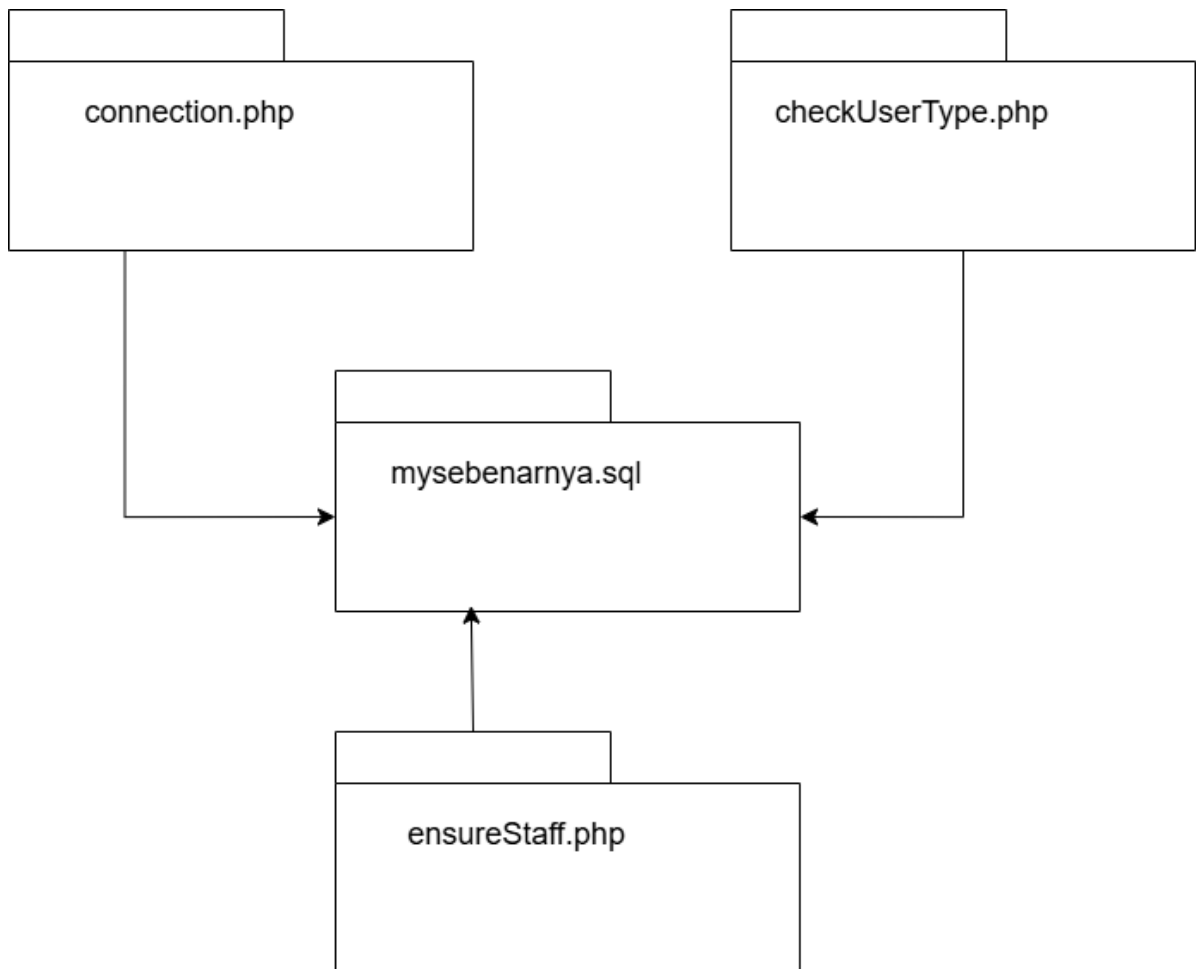


Figure 3.3 Middleware Layer for MySebenarnya System

Package Name	Description
connection.php	Create a connection to the database from the server.
mysebenarnya.sql	Establish a server to database connection.
checkUserType.php	Helps filter authentication requests based on user role.
EnsureStaff.php	Allows only Staff to approve and categorize.

3.4 PACKAGE RELATIONSHIP

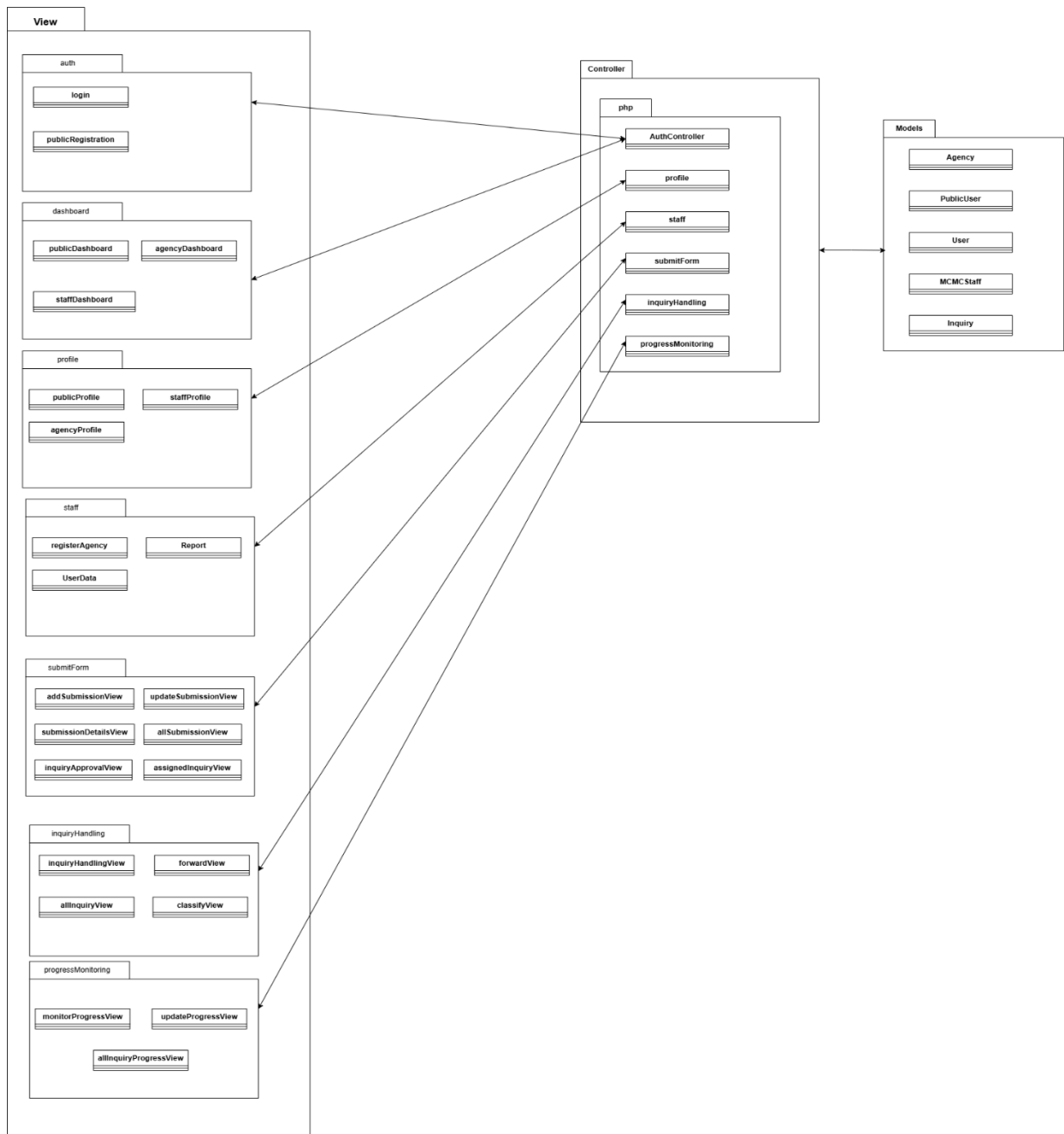


Figure 3.4 Model View Controller (MVC) Package Relationship for MySebenarnya Sistem

4. DETAIL DESIGN

4.1 MODEL [SDD_MYS_100]

4.1.1 User Model [SDD_MYS_100_101]

Class Type	Entity class	
Responsibility	A model class that represents system users and manages authentication.	
Attributes	Attributes Name	Attributes Type
	email	string
	password	string
	user_type	string
Methods	Method Name	Description
	publicUser()	HasOne relationship with PublicUser
	mcmcStaff()	HasOne relationship with McmcStaff
	agency()	HasOne relationship with Agency
	getUserType()	Determines user type by email domain
	isAdmin()	Checks if user is MCMC staff
Algorithm	1. Login process: a. Validate credentials b. Attempt authentication c. Update user_type if needed d. Redirect to dashboard 2. Registration process: a. Validate input b. Create user record c. Create public user profile d. Log user in 3. Logout process: a. Invalidate session	

	b. Regenerate token
--	---------------------

4.1.2 PublicUser Model [SDD_MYS_100_102]

Class Type	Entity class	
Responsibility	A model class that stores profile data for public users.	
Attributes	Attributes Name	Attributes Type
	publicName	string
	publicContact	string
	publicGender	string
	publicAge	integer
Methods	Method Name	Description
	user()	BelongsTo relationship with User
Algorithm	1. Data access: a. Return related user when accessed	

4.1.3 McmcStaff Model [SDD_MYS_100_103]

Class Type	Boundary class	
Responsibility	Stores profile data for MCMC staff	
Attributes	Attributes Name	Attributes Type
	name	string
	contact	string
	job_title	string
	department	string
Methods	Method Name	Description
	user()	BelongsTo relationship with User
Algorithm	1. Data access: a. Return related user when accessed	

4.1.4 Agency Model [SDD_MYS_100_104]

Class Type	Entity class	
Responsibility	Manages agency data and relationships in the system, including profile information, status tracking, and type classification	
Attributes	Attributes Name	Attributes Type
	name	string
	contact	string
	address	string
Methods	Method Name	Description
	user()	BelongsTo relationship with Use
Algorithm	1. Data access: a. Return related user when accessed	

4.1.5 ActivityLog Model [SDD_MYS_100_105]

Class Type	Entity class	
Responsibility	An interface for public user, mcmc staff, and agency to login to access to MySebenarnya system.	
Attributes	Attributes Name	Attributes Type
	email	String
	password	String
Methods	Method Name	Description
	@extends('layout.app')	Inherits from the app layout
	@section('title')	Defines the page title session
	@section('content')	Defines the main content session
	route('register')	Routes to store user registration data
	Route('login')	Routes to login page
Algorithm		

4.1.6 InquiryModel [SDD_MYS_100_201]

Class Type	Entity Class
------------	--------------

Responsibility	A model class that handles the storage and retrieval of inquiries submitted by public users, including metadata and relationships to user accounts.	
Attributes	Attributes Name	Attributes Type
	inquiryID	string
	publicID	string
	agencyID	string
	staffID	string
	inquiryTitle	string
	inquiryDescription	string
	inquiryProof	string
	inquiryStatus	string
	inquiryCategory	string
	inquiryFeedback	string
	dateSubmitted	timestamp
Methods	Method Name	Description
	publicUser()	Defines a belongsTo relationship to the public user.
	mcmcStaff()	Defines a belongsTo relationship to MCMC staff.
	agency()	Defines a belongsTo relationship to agency.
	isApproved()	Returns true if the inquiry has been approved.
	isPending()	Returns true if the inquiry is pending.
	hasFeedback()	Checks if the agency has provided any feedback and returns true if inquiryFeedback not null.
	scopeLatestSubmitted (\$query)	Returns inquiries ordered by the most recent dateSubmitted.

	scopeCategory(\$query, \$category)	A local query scope to filter inquiries by a specific category.
	scopeStatus(\$query, \$status)	A local query scope to filter inquiries based on their current status.
Algorithm	Start 1. Define the Inquiry model class that extends Model. 2. Define the relationship methods: a. publicUser() – returns the PublicUser linked to this inquiry. b. mcmcStaff() – returns the MCMCStaff who approved this inquiry. c. agency() – returns the Agency assigned to this inquiry. 3. Define helper functions: a. isApproved() – returns true if inquiryStatus equals 'Approved'. b. isPending() – returns true if inquiryStatus equals 'Pending'. c. hasFeedback() – returns true if inquiryFeedback is not null. 4. Define query scopes: a. scopeByCategory(\$query, \$category) – filters inquiries by inquiryCategory. b. scopeByStatus(\$query, \$status) – filters inquiries by inquiryStatus. c. scopeLatestSubmitted(\$query) – sorts inquiries by dateSubmitted in descending order. 5. When storing a new inquiry: a. Validate title and description. b. If a file is uploaded: i. Store the file in the public directory. ii. Save the file path to inquiryProof. c. Set inquiryStatus to 'Pending'. d. Set dateSubmitted to current timestamp. e. Assign publicID from the authenticated user. f. Save the new inquiry record. 6. When approving an inquiry: a. Retrieve the inquiry by ID. b. Set inquiryStatus to 'Approved'. c. Assign selected inquiryCategory. d. Assign staffID of the logged-in staff. e. Save the updated record.	

	7. When an agency submits feedback: a. Retrieve the inquiry assigned to the agency. b. Validate that feedback is not empty. c. Save the feedback in inquiryFeedback. d. Assign agencyID of the logged-in agency. e. Save the updated record. End
--	--

4.2 VIEW [SDD_MYS_200]

4.2.1 MANAGE USER [SDD_MYS_200_100]

4.2.1.1 Login UI [SDD_MYS_200_101]

Class Type	Boundary class	
Responsibility	An interface for public user, mcmc staff, and agency to login to access to MySebenarnya system.	
Attributes	Attributes Name	Attributes Type
	\$errors	collection
	old('email')	string
Methods	Method Name	Description
	@extends('layout.app')	Inherits from the app layout
	@section('title')	Defines the page title session
	@section('content')	Defines the main content session
	route('register')	Routes to store user registration data
	Route('login')	Routes to login page
Algorithm	1. Display: a. Show logo and title b. Display error messages if any c. Render email/password fields d. Provide register link	

4.2.1.2 publicDashboard UI [SDD_MYS_200_102]

Class Type	Boundary class	
Responsibility	A landing page interface for public user dashboard.	
Attributes	Attributes Name	Attributes Type
	\$user	User object
Methods	Method Name	Description

	@extends('layout.public')	Inherits from the app layout
	@section('title')	Defines the page title session
	@section('content')	Defines the main content session
	navigateTo()	Handles menu navigation
	Route('login')	Routes to login page
Algorithm	1. Display: a. Sidebar with user info and menu b. Main content with widgets c. Quick action buttons 2. Navigation: a. Update active menu item b. Redirect to appropriate route	

4.2.1.3 StaffDashboard UI [SDD_MYS_200_103]

Class Type	Boundary class	
Responsibility	Dashboard interface for MCMC staff.	
Attributes	Attributes Name	Attributes Type
	\$user	User object
	\$menu_items	array
Methods	Method Name	Description
	@extends('layout.app')	Inherits from the app layout
	@section('title')	Defines the page title session
	@section('content')	Defines the main content session
	navigateTo()	Handles menu navigation
	Route('login')	Routes to login page
Algorithm	1. Display: a. Sidebar with staff-specific menu b. Stats widgets c. Recent activity feed	

	2. Navigation: a. Special handling for agency registration
--	---

4.2.1.4 agencyDashboard UI [SDD_MYS_200_104]

Class Type	Boundary class	
Responsibility	A landing page interface for agency dashboard.	
Attributes	Attributes Name	Attributes Type
	\$user	User object
Methods	Method Name	Description
	@extends('layout.public')	Inherits from the app layout
	@section('title')	Defines the page title session
	@section('content')	Defines the main content session
	navigateTo()	Handles menu navigation
	Route('login')	Routes to login page
Algorithm	1. Display: d. Sidebar with user info and menu e. Main content with widgets f. Quick action buttons 2. Navigation: c. Update active menu item d. Redirect to appropriate route	

4.2.1.5 PublicProfile [SDD_MYS_200_105]

Class Type	Boundary class	
Responsibility	A page for public profile	
Attributes	Attributes Name	Attributes Type
	email	String
	password	String

Methods	Method Name	Description
	@extends('layout.app')	Inherits from the app layout
	@section('title')	Defines the page title session
	@section('content')	Defines the main content session
	route('register')	Routes to store user registration data
	Route('login')	Routes to login page
Algorithm		

4.2.1.6 StaffProfile [SDD_MYS_200_106]

Class Type	Boundary class	
Responsibility	An interface for public user, mcmc staff, and agency to login to access to MySebenarnya system.	
Attributes	Attributes Name	Attributes Type
	email	String
	password	String
Methods	Method Name	Description
	@extends('layout.app')	Inherits from the app layout
	@section('title')	Defines the page title session
	@section('content')	Defines the main content session
	route('register')	Routes to store user registration data
	Route('login')	Routes to login page
Algorithm		

4.2.1.7 AgencyProfile [SDD_MYS_200_107]

Class Type	Boundary class	
Responsibility	An interface for public user, mcmc staff, and agency to login to access to MySebenarnya system.	

Attributes	Attributes Name	Attributes Type
	email	String
	password	String
Methods	Method Name	Description
	@extends('layout.app')	Inherits from the app layout
	@section('title')	Defines the page title session
	@section('content')	Defines the main content session
	route('register')	Routes to store user registration data
	Route('login')	Routes to login page
Algorithm		

4.2.1.8 RegisterAgency UI [SDD_MYS_200_108]

Class Type	Boundary class	
Responsibility	An interface for public user, mcmc staff, and agency to login to access to MySebenarnya system.	
Attributes	Attributes Name	Attributes Type
	email	String
	password	String
Methods	Method Name	Description
	@extends('layout.app')	Inherits from the app layout
	@section('title')	Defines the page title session
	@section('content')	Defines the main content session
	Route('login')	Routes to login page
Algorithm		

4.2.1.9 UserData UI [SDD_MYS_200_109]

Class Type	Boundary class	
Responsibility	An interface for public user, mcmc staff, and agency to login to access to MySebenarnya system.	
Attributes	Attributes Name	Attributes Type
	email	String
	password	String
Methods	Method Name	Description
	@extends('layout.app')	Inherits from the app layout
	@section('title')	Defines the page title session
	@section('content')	Defines the main content session
	Route('login')	Routes to login page
Algorithm		

4.2.1.10 Report UI [SDD_MYS_200_110]

Class Type	Boundary class	
Responsibility	An interface for public user, mcmc staff, and agency to login to access to MySebenarnya system.	
Attributes	Attributes Name	Attributes Type
	email	String
	password	String
Methods	Method Name	Description
	@extends('layout.app')	Inherits from the app layout
	@section('title')	Defines the page title session
	@section('content')	Defines the main content session
	Route('login')	Routes to login page
Algorithm		

4.2.1.11 PublicRegistration UI [SDD_MYS_200_1011]

Class Type	Boundary class	
Responsibility	An interface for public user to register to access to MySebenarnya system.	
Attributes	Attributes Name	Attributes Type
	\$errors	collection
	old() values	various
Methods	Method Name	Description
	@extends('layouts.app')	Inherits app layout
	@section('content')	Defines main content
Algorithm	1. Display: - Show logo and title - Render all required fields - Include password confirmation - Provide login link	

4.2.2 MANAGE INQUIRY FORM SUBMISSION [SDD_MYS_200_200]

4.2.2.1 addSubmission View [SDD_MYS_200_201]

Class Type	View Class	
Responsibility	Allows public users to submit a new inquiry.	
Attributes	Attributes Name	Attributes Type
	inquiryTitle	string
	inquiryDescription	string
	inquiryProof	string (file)
Methods	Method Name	Description
	@section('form')	Displays form input fields.
	@error('inquiryTitle')	Handles validation error for title.
	@csrf	Adds Laravel CSRF token to form.
	submitForm()	Submit the form if confirmed.
Algorithm	Algorithm PublicUserAddSubmissionView: Input: inquiryTitle, inquiryDescription, inquiryProof Output: HTML structure for the addSubmission interface Step 1: Start Step 2: Define the HTML structure for the entire page, including the document type, language, and head section with metadata. Step 3: Define a script for the submitForm function to handle form submission. Step 4: Set the body style with attributes like height, width, background image and position. Step 5: Create an aside element for the sidebar: Step 5.1: Define a div for the logo, including an anchor tag linking to the dashboard and an image tag for the logo. Step 5.2: Create an unordered list to contain navigation links grouped by categories.	

	Step 6: Create an aside element for the topbar: Step 6.1: Define a div for the topbar. Step 6.2: Add a logout link to the right-hand side of the page. Step 7: Create a main element for the main content area: Step 7.1: Create an inquiry form asking input of the inquiry title, inquiry description and the inquiry proof from the user. Step 7.2: Add a “Submit” button at the end of the form with an onclick event to trigger the formSubmission function. Step 8: Save the inquiry submitted with inquiryStatus = ‘Pending’ and the dateSubmitted = current date. Step 9: Output the complete HTML structure. Step 10: End Algorithm.
--	--

4.2.2.2 submissionDetails View [SDD_MYS_200_202]

Class Type	Boundary Class	
Responsibility	Show full details of an inquiry submitted by the Public User.	
Attributes	Attributes Name	Attributes Type
	inquiryTitle	string
	inquiryDescription	string
	inquiryStatus	string
	dateSubmitted	timestamp
	inquiryProof	string (file)
Methods	Method Name	Description
	@section('form')	Displays form input fields.
Algorithm	Algorithm PublicUserSubmissionDetailsView: Input: Not applicable. Output: HTML structure for the review and submissionDetails interface.	

	Step 1: Start Step 2: Define the HTML structure for the entire page, including the document type, language, and head section with metadata. Step 3: Set the body style with attributes like height, width, background image and position. Step 4: Create an aside element for the sidebar: Step 4.1: Define a div for the logo, including an anchor tag linking to the dashboard and an image tag for the logo. Step 4.2: Create an unordered list to contain navigation links grouped by categories. Step 5: Create an aside element for the topbar: Step 5.1: Define a div for the topbar. Step 5.2: Add a logout link to the right-hand side of the page. Step 6: Create a main element for the main content area: Step 6.1: Retrieve all data about inquiry by inquiryID. Step 6.2: Display all related fields in a detailed format. Step 7: Output the complete HTML structure. Step 8: End Algorithm.
--	---

4.2.2.3 updateSubmission View [SDD_MYS_200_203]

Class Type	Boundary Class	
Responsibility	Allow the Public User to update an existing inquiry that has not yet been approved.	
Attributes	Attributes Name	Attributes Type
	inquiryTitle	string
	inquiryDescription	string
	inquiryProof	string (file)

Methods	Method Name	Description
	@method('PUT')	Spoofs the HTTP method as PUT for an update.
	@error('inquiryTitle')	Handles validation error for title.
	@csrf	Adds Laravel CSRF token to form.
	submitForm()	Submit the form if confirmed.
Algorithm	Algorithm PublicUserUpdateSubmissionView: Input: inquiryTitle, inquiryDescription, inquiryProof Output: HTML structure for the updateSubmission interface Step 1: Start Step 2: Define the HTML structure for the entire page, including the document type, language, and head section with metadata. Step 3: Define a script for the submitForm function to handle form submission. Step 4: Set the body style with attributes like height, width, background image and position. Step 5: Create an aside element for the sidebar: Step 5.1: Define a div for the logo, including an anchor tag linking to the dashboard and an image tag for the logo. Step 5.2: Create an unordered list to contain navigation links grouped by categories. Step 6: Create an aside element for the topbar: Step 6.1: Define a div for the topbar. Step 6.2: Add a logout link to the right-hand side of the page. Step 7: Create a main element for the main content area: Step 7.1: Create an inquiry form asking input of the inquiry title, inquiry description and the inquiry proof with placeholders from the previous user input.	

	Step 7.2: Add a “Submit” button at the end of the form with an onclick event to trigger the formSubmission function. Step 7.3: Save the inquiry submitted with inquiryStatus = ‘Pending’. Step 8: Output the complete HTML structure. Step 9: End Algorithm.
--	--

4.2.2.4 allSubmission View [SDD_MYS_200_204]

Class Type	View Class	
Responsibility	Displays a list of inquiries submitted by the logged in public user.	
Attributes	Attributes Name	Attributes Type
	inquiryID	string
	inquiryTitle	string
	inquiryStatus	string
	dateSubmitted	timestamp
Methods	Method Name	Description
	@foreach(\$inquiries as \$inquiry)	Iterates through each user inquiry.
	@section('content')	Contains the table layout for inquiries.
Algorithm	Algorithm PublicUserDeleteSubmissionView: Input: Not applicable. Output: HTML structure for the allSubmission interface. Step 1: Start Step 2: Define the HTML structure for the entire page, including the document type, language, and head section with metadata. Step 3: Set the body style with attributes like height, width, background image and position.	

	Step 4: Create an aside element for the sidebar: Step 4.1: Define a div for the logo, including an anchor tag linking to the dashboard and an image tag for the logo. Step 4.2: Create an unordered list to contain navigation links grouped by categories. Step 5: Create an aside element for the topbar: Step 5.1: Define a div for the topbar. Step 5.2: Add a logout link to the right-hand side of the page. Step 6: Create a main element for the main content area: Step 6.1: Retrieve all data about inquiry by publicID. Step 6.2: Display all related fields in a detailed format with “Edit”, “Delete” and “View” button at the side of each inquiry. Step 6.3: Link all the buttons to their respective pages, with an onclick function. Step 7: Output the complete HTML structure. Step 8: End Algorithm.
--	--

4.2.2.5 inquiryApproval Page [SDD_MYS_200_205]

Class Type	Boundary Class	
Responsibility	Allow MCMC Staff to approve pending inquiries.	
Attributes	Attributes Name	Attributes Type
	inquiryTitle	string
	inquiryDescription	string
	inquiryStatus	string
	dateSubmitted	timestamp
	inquiryProof	string (file)
Methods	Method Name	Description
	@method('PUT')	Submits the form as an update request.

Algorithm	Algorithm StaffInquiryApprovalView: Input: Not applicable. Output: HTML structure for the inquiryApproval interface. Step 1: Start Step 2: Define the HTML structure for the entire page, including the document type, language, and head section with metadata. Step 3: Set the body style with attributes like height, width, background image and position. Step 4: Create an aside element for the sidebar: Step 4.1: Define a div for the logo, including an anchor tag linking to the dashboard and an image tag for the logo. Step 4.2: Create an unordered list to contain navigation links grouped by categories. Step 5: Create an aside element for the topbar: Step 5.1: Define a div for the topbar. Step 5.2: Add a logout link to the right-hand side of the page. Step 6: Create a main element for the main content area: Step 6.1: Retrieve all data with the inquiryStatus = 'Pending'. Step 6.2: Display all related fields in a detailed format with a 'View Details', 'Approve' and 'Reject' button at the side. Step 6.2.1: Retrieve all data by the inquiryID selected. Step 6.2.2: Display all related fields in a detailed format. Step 6.3: Select 'Approve' or 'Reject' button. Step 7: Output the complete HTML structure. Step 8: End Algorithm.
-----------	--

4.2.2.6 assignedInquiries View [SDD_MYS_200_206]

Class Type	Boundary Class	
Responsibility	Allow Agency users to view and respond to inquiries assigned to them.	
Attributes	Attributes Name	Attributes Type
	inquiryTitle	string
	inquiryDescription	string
	inquiryStatus	string
	dateSubmitted	timestamp
	inquiryProof	string (file)
Methods	Method Name	Description
	@section('form')	Displays form input fields.
Algorithm	Algorithm AgencyAssignedInquiryView: Input: Not applicable. Output: HTML structure for the assignedInquiry interface. Step 1: Start Step 2: Define the HTML structure for the entire page, including the document type, language, and head section with metadata. Step 3: Set the body style with attributes like height, width, background image and position. Step 4: Create an aside element for the sidebar: Step 4.1: Define a div for the logo, including an anchor tag linking to the dashboard and an image tag for the logo. Step 4.2: Create an unordered list to contain navigation links grouped by categories. Step 5: Create an aside element for the topbar: Step 5.1: Define a div for the topbar. Step 5.2: Add a logout link to the right-hand side of the page. Step 6: Create a main element for the main content area:	

	Step 6.1: Retrieve inquiries assigned to the agency based on agencyID and inquiryStatus = 'Approved'. Step 6.2: Display all details in a detailed structure. Step 7: Output the complete HTML structure. Step 8: End Algorithm.
--	---

4.2.3 MANAGE INQUIRY ASSIGNMENT [SDD_MYS_200_300]

4.2.3.1 InquiryHandling View [SDD_MYS_200_301]

Class Type	Boundary class	
Responsibility	An interface for MCMC staff to view incoming inquiries submitted by public users.	
Attributes	Attributes Name	Attributes Type
	\$inquiryID	string
	\$publicID	string
	\$inquiryDescription	string
	\$dateSubmitted	date
	\$inquiryCategory	string
	\$agencyID	string
Methods	Method Name	Description
	@extends('layout.app')	Inherits from the app layout
	@section('title')	Sets the title to "Incoming Inquiries"
	@section('content')	Displays inquiries submitted by public users
	route('inquiry.details')	Routes to a detailed view of a selected inquiry
Algorithm	Algorithm: InquiryHandling View Input: Not applicable Output: HTML structure for the InquiryHandling interface Step 1: Start Step 2: Define the HTML structure for the entire page, including the document type, language, and head section with metadata. Step 3: Set the body style with attributes like height, width, background image and position. Step 4: Create an aside element for the sidebar:	

	Step 4.1: Define a div for the logo, including an anchor tag linking to the dashboard and an image tag for the logo. Step 4.2: Create an unordered list to contain navigation links grouped by categories. Step 5: Create an aside element for the topbar: Step 5.1: Define a div for the topbar. Step 5.2: Add a logout link to the right-hand side of the page. Step 6: Create a main element for the main content area: Step 6.1: Retrieve all inquiries where inquiryCategory IS NULL. Step 6.2: Display the following attributes in a table format: • inquiryID • publicID • inquiryDescription • dateSubmitted • inquiryCategory • agencyID Step 6.3: Provide action links to view or classify each inquiry. Step 7: Output the complete HTML structure. Step 8: End Algorithm
--	--

4.2.3.2 Classify View [SDD_MYS_200_302]

Class Type	Boundary class	
Responsibility	An interface for MCMC staff to classify an inquiry into a suitable category before it is forwarded.	
Attributes	Attributes Name	Attributes Type
	\$inquiryID	string
	\$publicID	string

	\$inquiryDescription	string
	\$dateSubmitted	date
	\$inquiryCategory	string
	\$agencyID	string
	\$inquiryStatus	string
Methods	Method Name	Description
	@extends('layout.app')	Inherits from the app layout
	@section('title')	Sets the title to "Classify Inquiry"
	@section('content')	Displays classification dropdown and inquiry details
	route('inquiry.classify')	Submits classification result and updates status
Algorithm	Algorithm: Classify View Input: inquiryID Output: HTML structure for the classify interface Step 1: Start Step 2: Define the HTML structure for the entire page, including the document type, language, and head section with metadata. Step 3: Set the body style with attributes like height, width, background image and position. Step 4: Create an aside element for the sidebar: Step 4.1: Define a div for the logo, including an anchor tag linking to the dashboard and an image tag for the logo. Step 4.2: Create an unordered list to contain navigation links grouped by categories. Step 5: Create an aside element for the topbar: Step 5.1: Define a div for the topbar.	

	Step 5.2: Add a logout link to the right-hand side of the page. Step 6: Create a main element for the main content area: Step 6.1: Retrieve inquiry data by inquiryID. Step 6.2: Display the following attributes in a form layout: • inquiryID • publicID • inquiryDescription • dateSubmitted • inquiryCategory • agencyID • inquiryStatus Step 6.3: Render a dropdown list to select an inquiry category. Step 6.4: Add a submit button to update the inquiry classification. Step 6.5: On submission, set inquiryCategory and inquiryStatus = 'Classified', then redirect to Forward View. Step 7: Output the complete HTML structure. Step 8: End Algorithm
--	---

4.2.3.3 Forward View [SDD_MYS_200_303]

Class Type	Boundary class	
Responsibility	An interface for MCMC staff to forward a previously classified inquiry to the appropriate agency.	
Attributes	Attributes Name	Attributes Type
	\$inquiryID	string
	\$publicID	string
	\$inquiryDescription	string

	\$dateSubmitted	date
	\$inquiryCategory	string
	\$agencyID	string
	\$inquiryStatus	string
Methods	Method Name	Description
	@extends('layout.app')	Inherits from the app layout
	@section('title')	Sets the title to "Forward Inquiry"
	@section('content')	Display agency dropdown and forward button
	route('inquiry.forward')	Submits the selected agency and updates inquiry status
Algorithm	Algorithm: Forward View Input: inquiryID Output: HTML structure for the forward interface Step 1: Start Step 2: Define the HTML structure for the entire page, including the document type, language, and head section with metadata. Step 3: Set the body style with attributes like height, width, background image and position. Step 4: Create an aside element for the sidebar: Step 4.1: Define a div for the logo, including an anchor tag linking to the dashboard and an image tag for the logo. Step 4.2: Create an unordered list to contain navigation links grouped by categories. Step 5: Create an aside element for the topbar:	

	Step 5.1: Define a div for the topbar. Step 5.2: Add a logout link to the right-hand side of the page. Step 6: Create a main element for the main content area: Step 6.1: Retrieve inquiry data where inquiryStatus = 'Classified'. Step 6.2: Display the following attributes in a form layout: • inquiryID • publicID • inquiryDescription • dateSubmitted • inquiryCategory • agencyID • inquiryStatus Step 6.3: Render a dropdown list to select the forwarding agency. Step 6.4: Add a submit button to confirm forwarding. Step 6.5: On submission, assign agencyID and staffID, set inquiryStatus = 'Forwarded', and display a success message. Step 7: Output the complete HTML structure. Step 8: End Algorithm
--	--

4.2.3.4 All Inquiry View [SDD_MYS_200_304]

Class Type	Boundary class	
Responsibility	An interface to display a full list of all inquiries submitted to the system.	
Attributes	Attributes Name	Attributes Type

	\$inquiryID	string
	\$publicID	string
	\$inquiryDescription	string
	\$dateSubmitted	date
	\$inquiryCategory	string
	\$agencyID	string
	\$inquiryStatus	string
Methods	Method Name	Description
	@extends('layout.app')	Inherits from the app layout
	@section('title')	Sets the title to "All Inquiries"
	@section('content')	Renders full list of inquiries in table or card layout
	route('inquiry.view')	Routes to full details of each inquiry
Algorithm	Algorithm: All Inquiry View Input: Not applicable Output: HTML structure for the allInquiry interface Step 1: Start Step 2: Define the HTML structure for the entire page, including the document type, language, and head section with metadata. Step 3: Set the body style with attributes like height, width, background image and position. Step 4: Create an aside element for the sidebar: Step 4.1: Define a div for the logo, including an anchor tag linking to the dashboard and an image tag for the logo.	

	Step 4.2: Create an unordered list to contain navigation links grouped by categories. Step 5: Create an aside element for the topbar: Step 5.1: Define a div for the topbar. Step 5.2: Add a logout link to the right-hand side of the page. Step 6: Create a main element for the main content area: Step 6.1: Retrieve all inquiries from the database. Step 6.2: Display the following attributes in a tabular layout: • inquiryID • publicID • inquiryDescription • dateSubmitted • inquiryCategory • agencyID • inquiryStatus Step 6.3: Provide filtering/search options by inquiryStatus, dateSubmitted, and inquiryCategory. Step 6.4: Add action links to view inquiry details via <code>route('inquiry.view')</code>. Step 7: Output the complete HTML structure. Step 8: End Algorithm
--	---

4.2.4 MANAGE PROGRESS TRACKING [SDD_MYS_200_400]

4.2.2.1 monitorProgress View [SDD_MYS_200_401]

Class Type	Boundary Class	
Responsibility	Allows public users to submit a new inquiry.	
Attributes	Attributes Name	Attributes Type
	\$inquiryID	string
	\$inquiryDescription	string
	\$dateSubmitted	date
	\$agencyName	string
	\$inquiryStatus	string
	\$inquiryFeedback	string
	\$inquiryProff	string(file)
Methods	Method Name	Description
	@section('content')	Displays the main content section with inquiry details.
	@foreach(\$inquiries)	Iterates and displays each user-submitted inquiry.
	@csrf	Adds Laravel CSRF token to form.

Algorithm	Input: inquiryID Output: HTML structure for the monitorProgress interface Step 1: Start Step 2: Define the HTML structure for the entire page, including the document type, language, and head section with metadata. Step 3: Define a script for displaying inquiry status dynamically or enhancing interactivity such as expanding status history. Step 4: Set the body style with attributes like height, width, background image and position. Step 5: Create an aside element for the sidebar: Step 5.1: Define a div for the logo, including an anchor tag linking to the dashboard and an image tag for the logo. Step 5.2: Create an unordered list to contain navigation links grouped by categories such as Dashboard, Add Inquiry, and Inquiry Status. Step 6: Create an aside element for the topbar: Step 6.1: Define a div for the topbar. Step 6.2: Add a logout link to the right-hand side of the page. Step 7: Create a main element for the main content area: Step 7.1: Retrieve inquiry data using the inquiryID and display details including inquiry title, description, agency name, date submitted, current status and feedback. Step 7.2: Display the inquiry proof as a downloadable or viewable file if it exists Step 7.3: Create a status history section that lists each status update with timestamp in a readable format. Step 8: Output the complete HTML structure. Step 9: End Algorithm.
-----------	--

4.2.2.1 allInquiryProgress View [SDD_MYS_200_402]

Class Type	Boundary Class	
Responsibility	Allows MCMC Staff to view all inquiry submitted by public user	
Attributes	Attributes Name	Attributes Type
	\$inquiryID	string
	\$inquiryDescription	string
	\$dateSubmitted	date
	\$agencyName	string
	\$inquiryStatus	string
	\$inquiryFeedback	string
	\$inquiryProff	string(file)
Methods	Method Name	Description
	@section('form')	Displays form input fields.
	@if(\$inquiry->status == 'Overdue')	Shows overdue alert popup
	@click('Notify')	Triggers email by send action to agency
	@csrf	Laravel CRSF protection
Algorithm	Input: none Output: HTML structure for the admin interface to monitor all inquiries Step 1: Start Step 2: Define the HTML structure for the entire page, including the document type, language, and head section with metadata. Step 3: Define a script for filtering, sorting, or notifying overdue inquiries. Step 4: Set the body style with attributes like height, width, background image and position. Step 5: Create an aside element for the sidebar: Step 5.1: Define a div for the logo, including an anchor tag linking to the admin dashboard and an image tag for the logo.	

	Step 5.2: Create an unordered list to contain navigation links grouped by categories such as Dashboard, Manage Inquiries, Reports, and Logout. Step 6: Create an aside element for the topbar: Step 6.1: Define a div for the topbar. Step 6.2: Add a logout link to the right-hand side of the page. Step 7: Create a main element for the main content area: Step 7.1: Retrieve and list all submitted inquiries from the database. Step 7.2: Display inquiry data in tabular format including inquiry ID, title, agency name, status, date submitted and options to view or escalate. Step 7.3: Provide action buttons to notify agencies for overdue or missing documentation inquiries. Step 7.4: Trigger popup modals or message boxes for confirmation of actions like “Notify Overdue” or “Send Email to Agency.” Step 8: Output the complete HTML structure. Step 9: End Algorithm.
--	---

4.2.2.1 updateProgress View [SDD_MYS_200_403]

Class Type	Boundary Class	
Responsibility	Allows agency to update the submitted inquiry.	
Attributes	Attributes Name	Attributes Type
	\$inquiryID	string
	\$inquiryDescription	string
	\$dateSubmitted	date
	\$agencyName	string
	\$inquiryStatus	string
	\$inquiryFeedback	string
	\$inquiryProff	string(file)

Methods	Method Name	Description
	@click('Status')	Submits the status update (to “In Progress” or “Resolved”)
	@click('Validity')	Submits the news validity (Real or Fake)
	@csrf	Laravel CSRF token for safe form submission
Algorithm	Input: inquiryID Output: HTML structure for the agency interface to update inquiry status Step 1: Start Step 2: Define the HTML structure for the entire page, including the document type, language, and head section with metadata. Step 3: Define a script for updating status, uploading documents, and submitting inquiry updates. Step 4: Set the body style with attributes like height, width, background image and position. Step 5: Create an aside element for the sidebar: Step 5.1: Define a div for the logo, including an anchor tag linking to the agency dashboard and an image tag for the logo. Step 5.2: Create an unordered list to contain navigation links grouped by categories such as Dashboard, View Assigned Inquiries, and Logout. Step 6: Create an aside element for the topbar: Step 6.1: Define a div for the topbar. Step 6.2: Add a logout link to the right-hand side of the page. Step 7: Create a main element for the main content area: Step 7.1: Retrieve the inquiry assigned to the agency using inquiryID and display its details. Step 7.2: Provide fields for the agency to update the inquiry status to either “In Progress” or “Resolved”.	

	Step 7.4: Add a validity dropdown or input to assess the news as valid or fake. Step 7.5: Add a submit button to update the inquiry in the system. Step 8: Output the complete HTML structure. Step 9: End Algorithm.
--	---

4.3 CONTROLLER [SDD_MYS_300]

4.3.1 MANAGE USER [SDD_MYS_300_100]

4.3.1.1 AuthController [SDD_MYS_300_101]

Class Type	Controller class	
Responsibility	Handles authentication processes including login, registration, and logout.	
Attributes	Attributes Name	Attributes Type
	credentials (email, password)	array
	user	User object
Methods	Method Name	Description
	showLogin()	Displays users login form view
	showRegister()	Displays public user registration form view
	login()	Authenticates user credentials
	register()	Processes new public user registration
	logout()	Handles user logout
Algorithm	Show Login Page: - Check if user is already logged in - If logged in, redirect to dashboard	

	c. If not logged in: i. Display login form with: - Email input field - Password input field - Login button - "Remember me" checkbox ii. Show error messages if previous login failed iii. Keep previously entered email filled in Process Login: a. Receive email and password from form b. Validate inputs: i. Email must contain @ symbol ii. Password must be at least 8 characters c. Check credentials against database: i. If match found: - Create login session - Redirect to appropriate dashboard based on user type ii. If no match: - Show "Invalid credentials" error - Return to login page with entered email Show Registration Page: a. Display registration form with: i. Full name field ii. Phone number field iii. Age dropdown (18-100) iv. Gender selection (Male/Female) v. Email field
--	--

	vi. Password and confirmation fields b. Show validation errors if previous submission failed Process Registration: a. Validate all form fields b. Check if email already exists c. If valid: i. Create new user account ii. Set default user type as "public" iii. Create public profile with: - Public display name - Contact number - Age - Gender iv. Log user in automatically v. Redirect to dashboard d. If invalid: i. Show specific error messages ii. Keep valid form data filled in Process Logout: a. End user session b. Clear authentication cookies a. c. Redirect to login page
--	---

4.3.1.2 ProfileController [SDD_MYS_300_102]

Class Type	Controller class	
Responsibility	Manages user profile viewing and updating	
Attributes	Attributes Name	Attributes Type
	user	User object
	user_type	string
Methods	Method Name	Description
	showProfile()	Displays profile view based on user type
	updateProfile()	Handles profile updates
	changePassword()	Processes password changes
Algorithm	1. Profile display: a. Check user type b. Return appropriate profile view 2. Profile update: a. Validate input b. Update corresponding profile model 3. Password change: a. Verify current password b. Validate new password c. Update password	

4.3.1.3 StaffController [SDD_MYS_300_103]

Class Type	Controller class	
Responsibility	Handles agency registration by staff	
Attributes	Attributes Name	Attributes Type
	agency_data	array
Methods	Method Name	Description
	showAgencyRegister()	Displays agency registration form
	registerAgency()	Processes agency registration
Algorithm	1. Profile display: a. Check user type b. Return appropriate profile view 2. Agency registration: a. Validate input (requires @agency.com email)	

	b. Create user record c. Create agency profile d. Redirect with success message
--	---

4.3.2 MANAGE INQUIRY FORM SUBMISSION [SDD_MYS_300_200]

4.3.2.1 SubmitForm Controller [SDD_MYS_300_201]

Class Type	Controller Class	
Responsibility	Handles all operations related to inquiry CRUD and approval flow.	
Attributes	Attributes Name	Attributes Type
	N/A	N/A
Methods	Method Name	Description
	index()	Returns all public inquiries for browsing.
	create()	Displays the add inquiry submission form.
	store()	Validate the request data and saves a new inquiry with inquiryStatus = 'Pending'.
	show(\$id)	Retrieves and displays a detailed view of the selected inquiry.
	edit(\$id)	Displays the edit form for the selected inquiry (if it is not yet approved).
	update(\$id)	Updates inquiry data in the database if valid.
	destroy(\$id)	Deletes an inquiry by inquiryID if the current user is the owner.
	myInquiries()	Retrieves all inquiries submitted by the logged-in public user.
	approve(\$id)	Updates inquiry status to 'Approved', assigns category and staffID.
	agencyInquiries()	Returns list of approved inquiries assigned to the current agency.

	submitFeedback(\$id)	Stores feedback from the agency in the inquiryFeedback field.
Algorithm	Process Add Inquiry: - Receive input from inquiry form (inquiryTitle, inquiryDescription, inquiryProof). - Validate inputs: - inquiryTitle is required. - InquiryDescription is required. - InquiryProof is required - If Valid: - Save inquiryStatus = "Pending". - Set dateSubmitted = current date - Link to current publicID - Redirect to allSubmission page. - If not valid: - Display validation errors. - Return to form with entered data. Process View Inquiry: - Receive inquiryID from request. - Retrieve all data of inquiry from the database. - Pass inquiry to view page for display. - If not found, return 404 error page. Process Edit Inquiry: - Ensure inquiry belongs to user and inquiryStatus = 'Pending'. - Show current data in editable form. - On submission, validate input. - If Valid:	

	i. Save inquiryStatus = “Pending”. ii. Redirect to allSubmission page. e. If not valid: i. Display validation errors. ii. Return to form with entered data. Process Delete Inquiry: a. Receive inquiryID. b. Confirm deletion of inquiryID. c. Delete inquiryID from database. d. Redirect to allSubmission page with success message. Process View All My Inquiries: a. Identify publicID of current user. b. Retrieve inquiries linked to that publicID. c. Display the data retrieved in a structured order. Process Inquiry Approval: a. Display all the inquiries that are pending. b. Identify the inquiryID that has been chosen by the Staff. c. Retrieve all the data of the chosen inquiryID. d. Display inquiryTitle, dateSubmitted, inquiryDescription and inquiryProof. e. If the Staff clicks on ‘Approve’ button: i. Set inquiryStatus = “Approved”. ii. Link to staffID that has approved the inquiry. iii. Save updates. iv. Redirect to the inquiryApproval page. f. If the Staff clicks on ‘Reject’ button: i. Set inquiryStatus = “Rejected”.
--	--

	ii. Link to staffID that has approved the inquiry. iii. Save updates. iv. Redirect to the inquiryApproval page. Process View Assigned Inquiries: a. Identify agencyID based on log in. b. Retrieve inquiryID where inquiryStatus = 'Approved' and agencyID that matches. c. Display all the data retrieved from the inquiryID.
--	---

4.3.3 MANAGE INQUIRY ASSIGNMENT [SDD_MYS_300_300]

4.3.3.1 InquiryCategory Controller [SDD_MYS_300_301]

Class Type	Controller class	
Responsibility	Manages inquiry processes including displaying inquiries, classifying them into categories, and forwarding them to the appropriate agencies.	
Attributes	Attributes Name	Attributes Type
	inquiryID	array
	inquiry	Inquiry object
	categoryList	array
	agencyList	array
	request	HTTP request object
Methods	Method Name	Description
	showAllInquiries()	Displays all submitted inquiries
	showUncategorized()	Displays inquiries that have not been categorized
	showClassifyForm()	Displays form for classifying a specific inquiry
	classifyInquiry()	Saves classification and updates inquiry status
	showForwardForm()	Displays form to forward a classified inquiry
	forwardInquiry()	Submits forwarding information and updates inquiry status
Algorithm	Show All Inquiries: a. Retrieve all inquiries from the database b. Display a list with attributes: • inquiryID • publicID • inquiryDescription • dateSubmitted • inquiryCategory • agencyID	

	• InquiryStatus c. Provide options to filter/search inquiries by category, status, and date Show Uncategorized Inquiries: a. Retrieve inquiries where inquiryCategory IS NULL b. Display the inquiry list with basic details c. Provide action buttons to classify or view inquiry details Show Classification Form: a. Receive inquiryID from user action b. Retrieve inquiry record by ID c. Retrieve list of predefined inquiry categories d. Display the following on the form: • inquiryID • inquiryDescription • dateSubmitted • Dropdown for inquiryCategory e. Show validation messages if previous classification failed Process Classification (classifyInquiry): a. Validate form submission: • Ensure inquiryID is valid • Ensure category is selected b. Update inquiry with: • inquiryCategory = selected category • inquiryStatus = 'Classified' c. Redirect user to the Forward Inquiry page for the same inquiryID Show Forward Form: a. Receive inquiryID from Classify flow b. Retrieve inquiry where inquiryStatus = 'Classified'
--	---

	c. Retrieve agencies associated with the selected category d. Display form containing: • Inquiry details (ID, description, category) • Dropdown to choose agencyID e. Show validation messages if forwarding failed before Process Forwarding (forwardInquiry): a. Validate submission: • Ensure agencyID is selected • Confirm inquiryStatus = 'Classified' b. Update inquiry with: • agencyID = selected agency • inquiryStatus = 'Forwarded' c. Redirect to confirmation or dashboard with success message
--	---

4.3.4 MANAGE PROGRESS TRACKING [SDD_MYS_300_400]

4.3.4.1 InquiryProgressController [SDD_MYS_300_401]

Class Type	Controller Class	
Responsibility	Handles all operations related to inquiry CRUD and approval flow.	
Attributes	Attributes Name	Attributes Type
	N/A	N/A
Methods	Method Name	Description
	publicStatus()	Displays the status page for public users to view the progress of their own inquiries.
	adminStatus()	Displays the full details and status history of the selected inquiry.
	markOverdue(\$id)	Validates and updates the status, document, and validity of the inquiry.
	notifyAgency(\$id)	Sends an email to notify the agency of overdue status.
	remindUpload(\$id)	Sends a reminder notification when the inquiry is resolved but missing files.
	viewStatusHistory(\$id)	Retrieves and displays the full status history of the inquiry.
	agencyUpdateStatus(\$id)	Displays the form for agency to update status and upload supporting files.
Algorithm	Process View Inquiry Progress (Public User): 1. Receive public user ID from session. 2. Retrieve all inquiries submitted by the user from the database. 3. For each inquiry, include inquiryTitle, inquiryDescription, inquiryStatus, agencyName, feedback, and submission date. 4. Pass inquiry data to the publicMonitorProgress view page for display.	

	5. If no inquiries found, display a message: “No inquiries submitted yet.” Process View All Inquiries (MCMC Staff): 1. Receive MCMC Staff session from request. 2. Retrieve all inquiries from the database. 3. Include inquiryTitle, inquiryStatus, agencyName, and date inquiry submitted. 4. If no inquiries found, return an empty table. Process Edit Inquiry Status (Agency): 1. Receive inquiryID from the request URL. 2. Retrieve the inquiry from the database. 3. Ensure inquiry is assigned to the currently logged-in agency. 4. Display current inquiry status, supportingDocument, and validity selector in a form. 5. If not found or unauthorized, return 404 or access denied page. Process Update Inquiry Status (Agency): 1. Receive input from status update form (inquiryStatus, supportingDocument, newsValidity). 2. Validate inputs: inquiryStatus is required. 3. If inquiryStatus is “Resolved”, supportingDocument is required. 4. If inquiryStatus is “Resolved”, newsValidity is required. 5. If valid: Save inquiry status to the database. 6. Upload and attach supportingDocument. 7. Update newsValidity value.
--	---

	8. Notify MCMC admin and public user of the update. 9. Redirect to the agency dashboard with success message. 10. If not valid: Display validation errors. 11. Return to form with entered data. Process Notify Overdue Inquiry (Admin): 1. Receive inquiryID from the request. 2. Check if inquiry has exceeded its resolution deadline and is not marked “Resolved”. 3. If overdue: Update inquiry status to “Overdue”. Display a confirmation popup: “Notify Agency?” 4. If confirmed: Send email to agency contact. Show success popup: “Overdue inquiry notification sent.” 5. If not confirmed: Return to adminStatus page with no changes. Process Remind Missing Document (Admin): 1. Receive inquiryID from the request. 2. Check if inquiryStatus is “Resolved” and supportingDocument is missing. 3. If true: Trigger popup to notify agency of missing documentation. 4. Display contact form for admin to send reminder email. 5. Send email and display success popup: “Reminder sent to agency.” 6. If supportingDocument exists: Display message: “Supporting document already uploaded.”
--	---

5. REQUIREMENT TRACEABILITY

5.1 MANAGE USER

Requirement ID	Description	Design ID
REQ_UC01_001	User Login – The system shall allow multiple user login into MySebenarnya system.	SDD-MYS-2025-101
REQ_UC01_002	Public Dashboard – The system shall provide public users with an overview of their account and quick access to services.	SDD-MYS-2025-102
REQ_UC01_003	Staff Dashboard – The system shall allow MCMC Staff to manage user registrations, view reports, and access administrative tools.	SDD-MYS-2025-103
REQ_UC01_004	Agency Dashboard - The system shall allow agencies to manage their profile, services, and related activities.	SDD-MYS-2025-104
REQ_UC01_005	Public Profile - The public users can view and edit their personal details including their password and security settings.	SDD-MYS-2025-105
REQ_UC01_006	Staff Profile - The MCMC Staff can view their personal details and change	SDD-MYS-2025-106

	their password recovery via email.	
REQ_UC01_007	Agency Profile - The agency can view and edit their personal details including their password and security settings.	SDD-MYS-2025-107
REQ_UC01_008	Agency Registration – The system shall allows MCMC Staff to register a new agency into Mysebenarnya system.	SDD-MYS-2025-108
REQ_UC01_009	View Users – The system shall allow MCMC Staff to view all registered user data within the system including their profiles, registration details, and activity logs.	SDD-MYS-2025-109
REQ_UC01_010	Report and Analysis – The system shall allow MCMC Staff to generate reports and the reports are downloadable.	SDD-MYS-2025-110
REQ_UC01_011	Public User Registration – Public user must register to the system to access the system.	SDD-MYS-2025-111

5.2 MANAGE INQUIRY FORM SUBMISSION

Requirement ID	Description	Design ID
REQ_UC02_001	Submit Inquiry - The system shall allow public users to submit a new inquiry using a specified form.	SDD-MYS-2025-201
REQ_UC02_002	View Past Inquiries – The public users shall be able to view a list of their previously submitted inquiries.	SDD-MYS-2025-202
REQ_UC02_003	Access to Public Inquiries - The public users can view all inquiries submitted by other individuals without disclosing their personal information.	SDD-MYS-2025-203
REQ_UC02_004	View New Inquiries - The MCMC Staff must access the newly submitted inquiries from public users.	SDD-MYS-2025-204
REQ_UC02_005	Inquiry Filtering – The MCMC Staff must review and filter inquiries before forwarding them to the appropriate agencies.	SDD-MYS-2025-205
REQ_UC02_006	Access to Previous Inquiries – The MCMC Staff shall be able to view a record of all previously filtered inquiries.	SDD-MYS-2025-206

REQ_UC02_007	Access to Previous Inquiries – The Agencies shall be able to view a record of all past inquiries assigned to them.	SDD-MYS-2025-207
REQ_UC02_008	Inquiry History - The Agencies should be able to review the full history of the inquiries assigned to them.	SDD-MYS-2025-208

5.3 MANAGE INQUIRY ASSIGNMENT

Requirement ID	Description	Design ID
REQ_UC03_001	Dashboard (MCMC) – MCMC staff can access dashboard with links to inquiries	SDD-MYS-2025-301
REQ_UC03_002	View Incoming Inquiries – Staff can view uncategorized public inquiries	SDD-MYS-2025-302
REQ_UC03_003	Classify Inquiry – Staff can categorize an inquiry before forwarding	SDD-MYS-2025-303
REQ_UC03_004	Forward Inquiry – Staff can forward a classified inquiry to a relevant agency	SDD-MYS-2025-304
REQ_UC03_005	View All Inquiries – Staff can view all inquiries regardless of status	SDD-MYS-2025-305

REQ_UC03_006	Submit Inquiry – Public user can submit a new inquiry with title, desc, proof	SDD-MYS-2025-306
REQ_UC03_007	View Submission History – Public user can view their own past inquiries	SDD-MYS-2025-307
REQ_UC03_008	Dashboard (Agency) – Agency user can view dashboard showing received inquiries	SDD-MYS-2025-308
REQ_UC03_009	View Forwarded Inquiries – Agency can view inquiries forwarded to them	SDD-MYS-2025-309

5.4 MANAGE PROGRESS TRACKING

Requirement ID	Description	Design ID
REQ_UC04_001	Dashboard (Public User) – Public user clicks “Inquiry Status” to view the status of their submitted inquiries, including status and updates.	SDD-MYS-2025-401
REQ_UC04_002	Dashboard (MCMC Staff) – MCMC Staff views dashboard with status of all inquiries submitted by the public, categorized by status and action required.	SDD-MYS-2025-402
REQ_UC04_003	Dashboard (Agency) – Agency users view assigned	SDD-MYS-2025-403

	inquiries, filter them, and track action deadlines.	
REQ_UC04_004	Update Progress – Agency updates complaint progress	SDD-MYS-2025-404
REQ_UC04_005	Upload Proof of Action – Agency uploads files PDF or image as evidence of action.	SDD-MYS-2025-405
REQ_UC04_006	Verify Complaint Completion – MCMC Staff reviews agency action and marks complaint as resolved.	SDD-MYS-2025-406
REQ_UC04_007	Progress Notification (Missing Docs) – System alerts MCMC Staff if agency fails to upload documents, with Message Box notification.	SDD-MYS-2025-407
REQ_UC04_008	Email Agency (Reminder) – MCMC Staff clicks button to email agency a reminder to act on overdue inquiries.	SDD-MYS-2025-408